

Student Details from APAAR ID

Tech Application Program Interface v 1.0

Contents

1 Introduction	
1.1 Document Objective	
2 Get Authorization Token API.....	
2.1 CURL	
2.2 URL	
2.3 Request Type	
2.4 Request Headers.....	
2.5 Request.....	
2.6 Response	
2.7 Response Code.....	
3 Get details by APAAR ID	
4.1 CURL	
4.2 URL	
4.3 Request Type	
4.4 Request Headers.....	
4.5 Request.....	
4.6 Response	
4.7 Response Code.....	

1 Introduction

This document lists all the APIs used in the system for Academic Bank of Credits (ABC).

1.1 Document Objective

The document aims to explain methods, parameters and models that are tightly integrated into the server code, allowing APIs to always stay in sync with the system.

2 Get Authorization Token API

This Web Service will be used to get Authorization Token for ABC APIs.

2.1 CURL

```
curl --location --request POST 'https://nadapi.digilocker.gov.in/v1/oauth' \  
--form 'customer_id=" "' \  
--form 'customer_secret_key=""'
```

2.2 URL

<https://nadapi.digilocker.gov.in/v1/oauth>

2.3 Request Type

POST

2.4 Request Headers

None

2.5 Request

```
customer_id:  
customer_secret_key:
```

2.6 Response

```
{
  {
    "access_token":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOlwvXC9uZC1kZXZhcGkuZGw2LmluXC92
MVwvb2FldGgiLCJpYXQiOiJl2OTI5ODYsImV4cCI6MTcyMjY5NjU4NiwiImJmIjozNzIyNjkyOTg2LCJqdG
kiOiJjWXM2WEpBWFlTEhWaFlYIiwic3ViIjojoiNjZhYzYlNTZhMDhkYWZjOWUyM2Y4N2I2IiwicHJ2IjojoiM2Qz
YTY4Nzk2MjEzM2MwOWI1ZWE5MDUzNWUwYjMxYmJjODliMjAyMCJ9.q-
Xeno7nRpLdob6r4uOoHL7daANMp5w6VquFQcABKq4",
    "token_type": "bearer",
    "expires_in": "60 mins"
  }
}
```

2.7 Response Code

Following table explains the various response codes:

HTTP Code	Description
200	Successful Response
400	Bad Request/Validation Error

3 Get Details From APAAR ID

This Web Service will be used to retrieve Student Details from Apaar Id

3.1 CURL

```
curl --location --request POST 'https://nadapi.digilocker.gov.in/v1/VerifyApaar' \
--header 'X-APISETU-APIKEY: ' \
--header 'Content-Type: application/json' \
--header 'Authorization: ..... ' \
--data '{
  "encryptedApaarData":
    "Vhh/NxA+QPZ0tv08sWLEfOSvqZjIE+XLbTljqW8DfcWO9m0jYPttwnikZdwjdpWexYThauxnPr1h5wtCVsEcK
+zEXOCLzEQWOj4fsUxRUu51dy5p3gdh56ldWIqdxALWzVzNNcBFyMFyBjgfJswQuDJ199kASEPJw5dEV9pIaI9
APER+PlgCEGJMYPVnQONRmYJGq05I28VCQDfuaUvCyKTIzdsd0D5jGxjlHb001a+ZuFTbPcPZmSTQBBKJyOFIs
z/y6i5ZE3Jdx3PaR5fzloNBqxIUCHVDuyQZMZpY/VZ57/DotWPiIJBnlMzBcFzgs4O3lR1FZy7/oHYuGI9WEWJ
VU1MnIqSmfDpaW9MEjVMsFX+KIQyJ4I8HdA1N7/PWNIM73q/UY1lVAw/MyWL9aw42e0CwIZuheJBVPwmHuWtTi
e0DyeMBwCpkzjx+Di8Td5a3Tve2SqDoCkY01T7cW7oHFJniRqkZGLS37iHR3DQDo3Ag/jhYA+UfjgEgGzbelmi
8L0kiBgeYwzH10XWIZxsmBff/WuS/a2sD9AF+CZ1d/BDiu2hIO4MX2N4lPXjcnFwNfn3dAQlYz7UfbpBgpYJjC
+JPUG7lr9pi6RUesoMyLtFsh9nMY+oSy+ISNwns7GmKuQrCAvPHYhPc+0kQillUftofTcc1MEfzKOESCAAnA+ly
KVj4O563w/6gbC/CadAdalJAjHZuLBuzWn77cp3jsLR2Kvm7OfFGA0agRp955C9pFEAU8b/QyPlCKrtH0vlte9
kY60QU6bubhidlk6g=",
    "signature": "erB/ =",
    "publickey": "8VLxvXGGGcbqZkrhG9GzWYPZ+ "
}'
```

Note :
signature and publickey not mandatory .

3.2 URL

<https://nadapi.digilocker.gov.in/v1/VerifyApaar>

3.3 Request Type

POST

3.4 Request Headers

Authorization:Bearer

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJodHRwOlwvXC9uYWVhZGlnaWxvY2tici5nb3YuaW5cL3YxXC9vYXV0aCIsImhhdCI6MTY3MzYxMDMwMywiZXhwIjoxNjc3NjEzOTAzLCJyYmYiOiE2NzMTAzMDMsImp0aSI6Imx3Ulcyb3NTMnpRWHpxb2giLCJzdWliOiI2MDg3ZmM3YzNkZDFiYjMwM2QyNmQ3ODMiLCJwcnYiOiIzZDNhNjg3OTYyMTMzYzA5YjVlYTkwNTM1ZTBiMzFiYmM4OWIyMDIwLn0.8dl16tUoc44-gxoxidabBujwP8H55oLVQ0k61xbJNjg

3.5 Request

Encryption and Decryption Data Method:

Language Use (PHP 7.4.2)

```
<?php
```

```
function encrypt($data, $key) {
    $cipher = "aes-256-ecb";
    $encryptedData = openssl_encrypt($data, $cipher, $key, OPENSSL_RAW_DATA);
    return base64_encode($encryptedData);
}

function decrypt($encryptedData, $key) {
    $cipher = "aes-256-ecb";
    $encryptedData = base64_decode($encryptedData);
    $decryptedData = openssl_decrypt($encryptedData, $cipher, $key, OPENSSL_RAW_DATA);
    return json_decode($decryptedData, true); // Convert JSON string to associative array
}
```

```
$key = "gEreuS9bpPLoGjqt1CDfkRMdckYhI6IY";
$json_data = '{
    "apaar_id": "213486101552",
    "aadhaar_name": "Ayush Kathpalia",
    "year_of_birth": "1997",
    "gender": "M",
    "txn_id": "11111",
    "is_provider_present": "true",
    "provider_artifact": {
        "provider": {
            "name": "Anjali Kumari",
            "authentication_mode": "UID",
            "authentication_id_no": "266561685034",
            "consent_relation": "Guardian",
            "consent_date": "04/07/2024",
            "consent_time": "",
            "consent_place": "Delhi"
        }
    }
}';
```

```

// Decode JSON data to PHP array
$data_array = json_decode($json_data, true);

// Add the "payload_time" key with the current date and time
$data_array['payload_time'] = date('Y-m-d H:i:s');

// Encode PHP array back to JSON
$updated_json_data = json_encode($data_array, JSON_PRETTY_PRINT);

// Encrypt the data
$encrypted_data = encrypt($updated_json_data, $key);
echo $encrypted_data . "\n";

$privateKey = <<<EOD
-----BEGIN RSA PRIVATE KEY-----
ABC
-----END RSA PRIVATE KEY-----
EOD;

// Create a new key resource from the private key
$privateKeyResource = openssl_pkey_get_private($privateKey, 'your_password_here');

if ($privateKeyResource === false) {
    die('Invalid private key or incorrect passphrase');
}

// Create a signature for the data
openssl_sign($updated_json_data, $signature, $privateKeyResource,
OPENSSL_ALGO_SHA256);

// Free the key resource
openssl_free_key($privateKeyResource);

// Encode the signature in base64 for storage or transmission
$base64Signature = base64_encode($signature);
echo "<br><br><br>";
echo $base64Signature;
// Decrypt data
$decryptedData = decrypt($encrypted_data, $key);

// Decrypt data
$decryptedData = decrypt($encrypted_data, $key);

// Decrypt data
$decryptedData = decrypt($encrypted_data, $key);

echo "<br><br><br>";
echo "Decrypted Data:\n";
print_r($decryptedData);

```

```
// Public Key
$json_data_key = '{
    "publickey": "-----BEGIN PUBLIC KEY-----\nXYZ \n-----END PUBLIC KEY-----"
}';

// Decode JSON data to PHP array
$data_array_key = json_decode($json_data_key, true);
$updated_json_data = json_encode($data_array_key, JSON_PRETTY_PRINT);
// Encrypt the public key using AES (for demo purposes)
$encrypted_public_key = encrypt($updated_json_data, $key);
echo 'Encrypted PUBLIC KEY:'. $encrypted_public_key . "\n";
?>
```

3.6 Response

Sucess Response :

```
{
    "status": 1,
    "statuscode": "200",
    "message": "Records Found Successfully",
    "ABC_ACCOUNT_ID": "608860222037",
    "CNAME": "Ayush Kathpalia",
    "GENDER": "M",
    "DOB": "08/02/2000",
    "CREATED_DATE": "2021-07-29 12:10:02",
    "UNIVERSITY_NAME": "NA",
    "COURSE_NAME": "NA",
    "PROGRAM_NAME": "NA",
    "ENROLLMENT_NO": "NA",
    "ROLL_NO": "NA",
    "REGN_NO": "NA"
}
```

Fail Response :

```
{
    "status": "fail",
    "status_code": "400",
    "message_code": "E-313",
    "txn_id": "",
    "date_time": "30/07/2024 16:24:54",
    "message": "Mandatory fields missing: TXN ID "
```



```
}
```

```
{  
  "status": "error",  
  "status_code": "400",  
  "txn_id": " ",  
  "date_time": "30/07/2024 16:23:14",  
  "message_code": "E-312",  
  "message": "Provider field must be true."  
}
```

```
{  
  "status": "fail",  
  "status_code": "400",  
  "message_code": "E-313",  
  "txn_id": "167262512",  
  "date_time": "30/07/2024 16:01:44",  
  "message": "Mandatory fields missing: PROVIDER NAME "  
}
```

```
{  
  "status": "fail",  
  "status_code": "400",  
  "message_code": "E-004",  
  "date_time": "30/07/2024 16:00:04",  
  "message": "Error in decrypting input data."  
}
```

```
{  
  "status":0,  
  "status_code":"400",  
  "message":"Records not found",  
  "txn_id":"11111",  
  "date_time":"05/08/202413:02:10",  
  "apaar_id":"213486101552"  
}
```

```
{  
  "status":"fail",  
  "status_code":"400",  
  "message_code":"E-313",  
  "txn_id":"11111",  
  "date_time":"05/08/2024 13:34:29",  
  "message":"Mandatory fields is missing: APAAR ID"  
}
```

```
{
  "status": "fail",
  "status_code": "400",
  "txn_id": "11111",
  "date_time": "05/08/2024 13:37:11",
  "message_code": "E-308",
  "message": "The Provisional ID must be 12 numeric digits with no repeated ."
}

{
  "status": "fail",
  "status_code": "400",
  "txn_id": "XyzTemp123456789",
  "date_time": "07/08/2024 11:38:20",
  "message_code": "C-306",
  "message": "Student Year of birth must be between 1950 and 2024."
}

{
  "status": "fail",
  "status_code": "400",
  "txn_id": "XyzTemp123456789",
  "date_time": "07/08/2024 11:36:46",
  "message_code": "C-306",
  "message": "Student Year of birth must be a four-digit and format should be YYYY."
}

{
  "status": "fail",
  "txn_id": "11111",
  "date_time": "05/08/2024 13:46:14",
  "status_code": "400",
  "message_code": "C-301",
  "message": "StudentGender field must be one of: M, F, T."
}

{
  "status": "error",
  "status_code": "400",
  "txn_id": "11111",
  "date_time": "05/08/2024 14:30:56",
  "message_code": "E-312",
  "message": "Consent Provider Field must be true."
}
```

```

{
    "status": "fail",
    "status_code": "400",
    "message_code": "E-313",
    "txn_id": "11111",
    "date_time": "05/08/2024 14:31:40",
    "message": "Mandatory fields missing: CONSENT PROVIDER NAME "
}

```

```

{
    "status": "fail",
    "status_code": "400",
    "message_code": "E-005",
    "date_time": "05/08/2024 12:38:45",
    "message": "Error in encrypting input data."
}

```

3.7 Response Code

Following table explains the various response codes:

HTTP Code	Description
200	Successful Response
400	(*) Mandatory field missing/Records not found
404	Input detail does not match with existing APAAR data.
500	Internal Server Error
